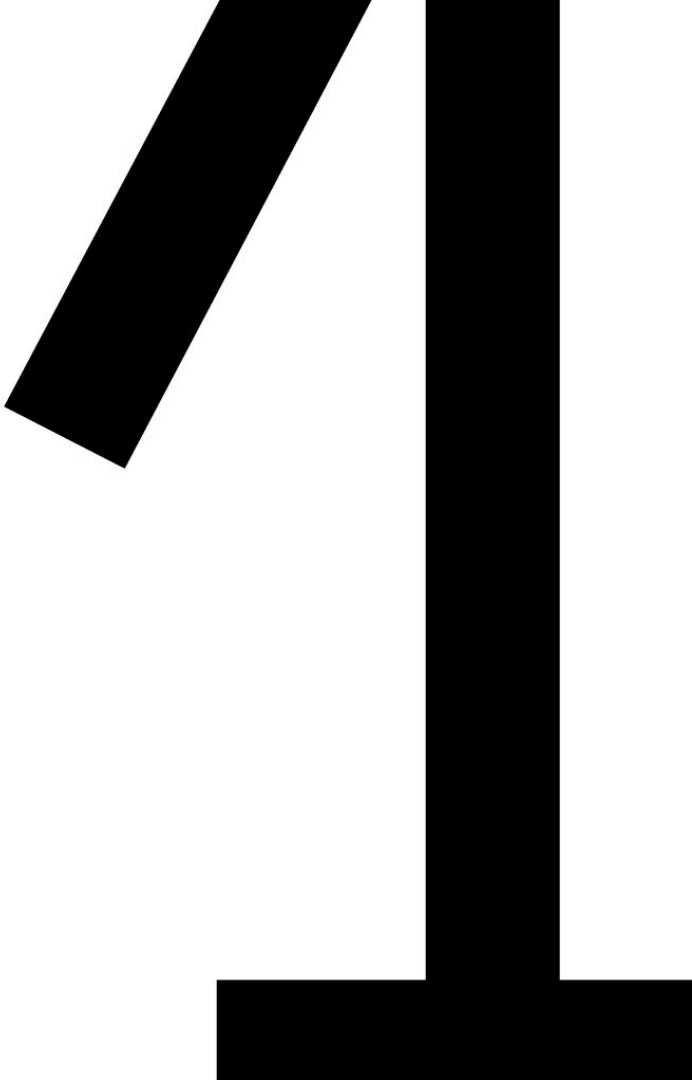
Fine Tune your DCD

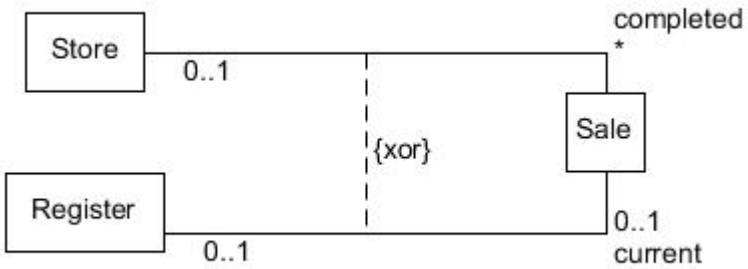
KdG Karel de Grote Hogeschool

Other additions DCD Constraints & stereotypes

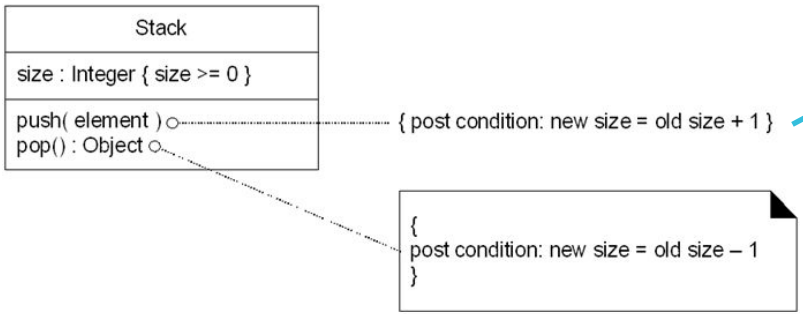


Constraints: logical condition

- A logical condition that elements must meet
- Possible notations:



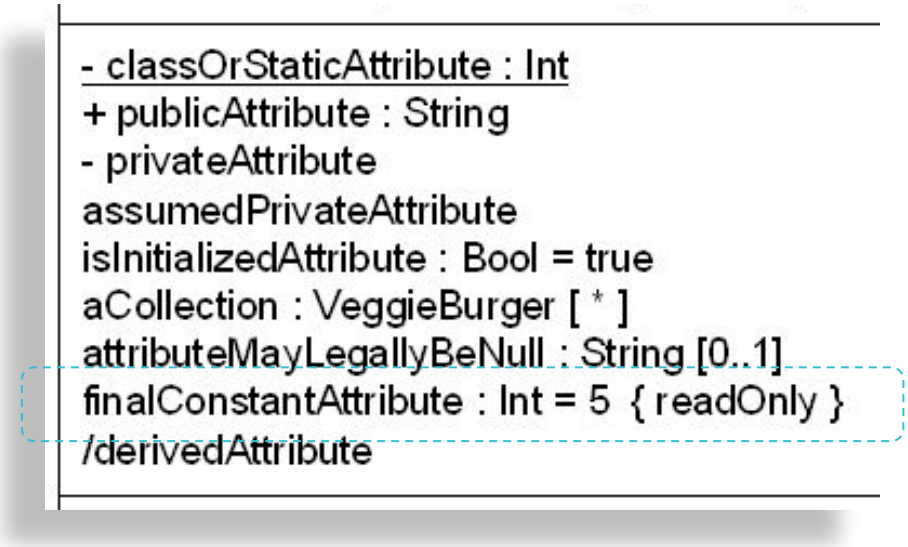
xor: exclusive or.
A sale is always connected to
a Store OR with a register



Always between {braces}

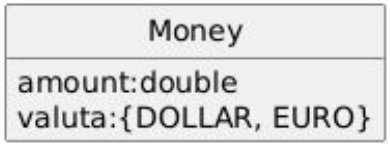
Constraints: property

- A UML keyword that specifies a property of an element



Constraints: limited number of values

- How do you implement the “currency” attribute?



- 2 possibilities according to UML that are similar:

- the 2nd shows a better implementation
- Use the explicit <<enumeration>> especially if it appears in multiple attributes

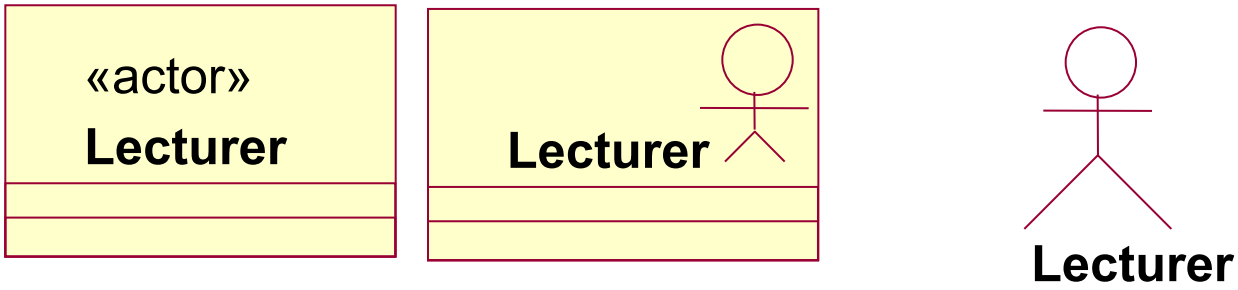
By the way:
NEVER use "double" for monetary amounts; it's a binary (base-2) representation that causes rounding errors. Compare it to our base-10 representation, which can never correctly represent 1/3. Use BigDecimal in Java. Look up how it works...

```
double price1 = 0.1;
double price2 = 0.2;
if (price1 + price2 == 0.3) {
  System.out.println("It's 0.3!"); // NEVER executed code!
} else {
  System.out.println("It's NOT 0.3!"); // Printed.
}
```

Stereotypes

- an enumeration is a variant of a normal class
  - every enumeration constant is an immutable object
- Variants of UML elements are indicated by a stereotype
- not every name between the 'fishbones' is a stereotype, 'actor' and 'dataType' are fundamental native elements and 'include' and 'extend' are keywords that specify the type of relationship between use cases
- may contain additional constraints and other extensions, such as a different graphical representation
- 3 Ways to Present Stereotypes

"name" decoration icon



Quotation marks « »

Guillemets (" ") are the official marking mechanism for stereotypes, but it's crucial to understand that in practice they're used for different purposes. This can cause confusion. We can divide the terms between guillemets into four main categories:

1. Formal stereotypes

These are the "real" stereotypes. They extend an existing UML element to give it a new, specific meaning. A stereotype therefore defines a new kind of element.

- **<<Create>> and <<Destroy>>: Stereotypes for a message in a Sequence Diagram. They specify that the message creates or destroys an object.**
- **<<Used>>: A stereotype for a dependency relationship. It indicates that one element needs another element to function.**
- **<<enumeration>>: A stereotype that specifies a class or datatype as an enumeration of immutable constants.**
- **<<singleton>>: A stereotype you apply to a class**

2. Contextual Keywords

These terms use guillemet notation out of convention, but are not formally stereotypes. They do not transform the element into a "new kind" of element, but add descriptive information or context.

- **<<Include>> and <<Extend>>: Keywords that specify the nature of the relationship between two Use Cases. They describe the relationship, not the Use Cases themselves.**
- **«local», «global», «parameter»: Keywords that describe the scope of an object instance in a DCD (locally created, globally available, or passed as a parameter).**

3. Referencing Native Elements (Source of Confusion) = Label

Sometimes, guillemet notation is used incorrectly or out of habit to refer to fundamental, native UML elements. These are not stereotypes.

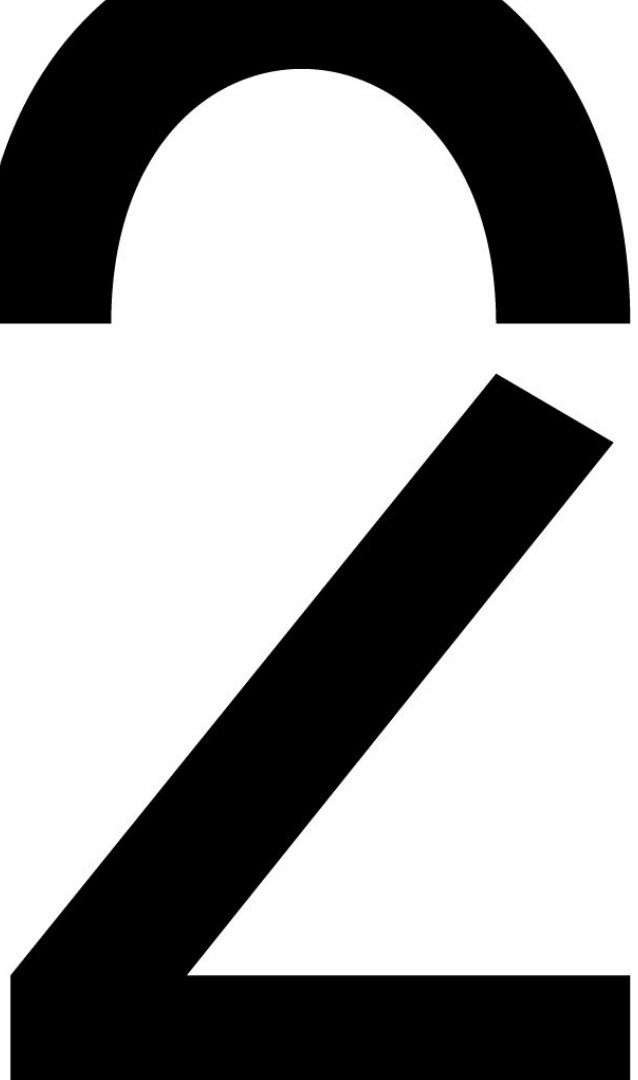
- **<<actor>>: An Actor is a fundamental UML element with its own symbol. It is not a stereotype of a class.**
- **«dataType»: A dataType, like a class, is a fundamental type classifier in UML.**
- **«interface»: An Interface, like a class, is a fundamental native element in UML.**

4. Modifier keywords

These keywords use guillemet notation as an alternative to another standard notation (such as italic text).

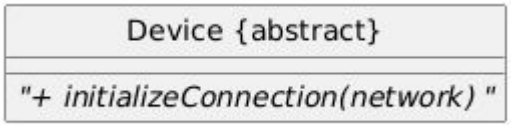
- **<<abstract>>: Indicates that a class or method cannot be directly instantiated. The default notation is italics.**
- **«static»: This keyword, applied to an attribute or an operation, indicates that it belongs to the class itself, rather than to an instance of the class.**

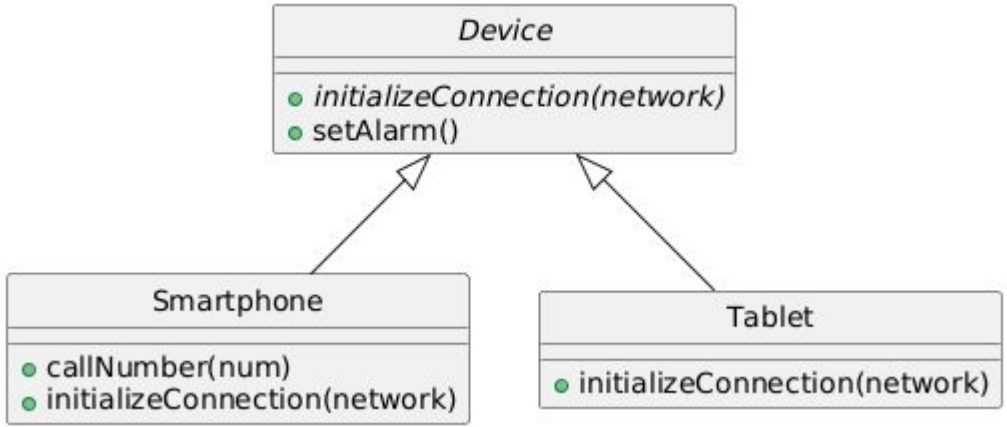
Other additions DCD Abstract classes & interfaces



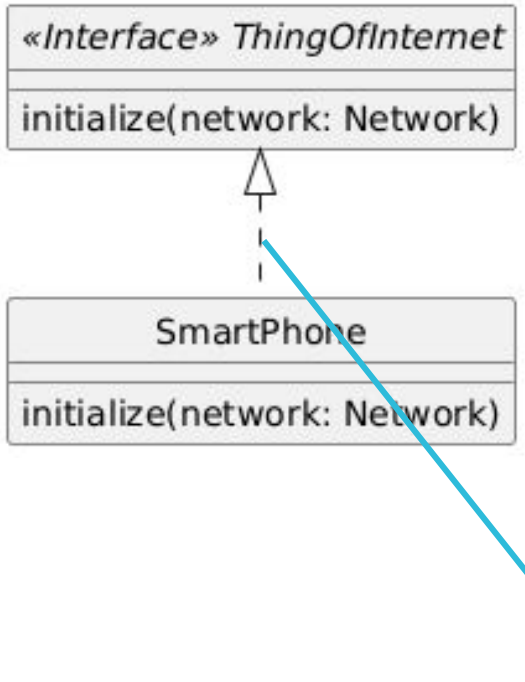
Abstract classes

- Class that cannot be instantiated.
- Contains (usually) one or more abstract operations
  - **Abstract operation**
    - Has no implementation
    - Implementation is done in the subclass
  - **Notation**
    - Digital diagram: italic
    - Diagram on paper: {abstract}



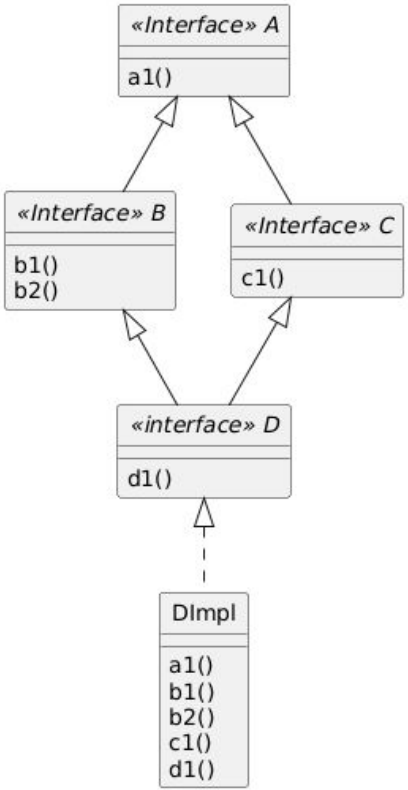


Interfaces



- contract, agreement that defines how you use an object
  - what methods can be called, what operations the class has
- Has no instance attributes
- Notation:
  - keyword «interface»
    - Between subclass and interface a generalisation arrow but with dashed line (implements)
    - Between subinterface and superinterface: An ordinary generalisation arrow with full line (extends)
  - For a class implementing the interface, you may also use the lollipop stereotype icon

Interfaces



- Notation:
  - keyword «interface»
    - Between subclass and interface a implementation arrow with a dashed line (implements)
    - Between subinterface and superinterface: A regular generalization arrow with solid line (extends)
  - For a class that implements the interface you may also use the lollipop alternative notation
  - If you iterate over an inherited method, it means you overwrite it (see polymorphism from analysis)
  - Note the distinction between the dotted lines and the solid arrows

Interfaces

Rules for Interfaces and Classes:

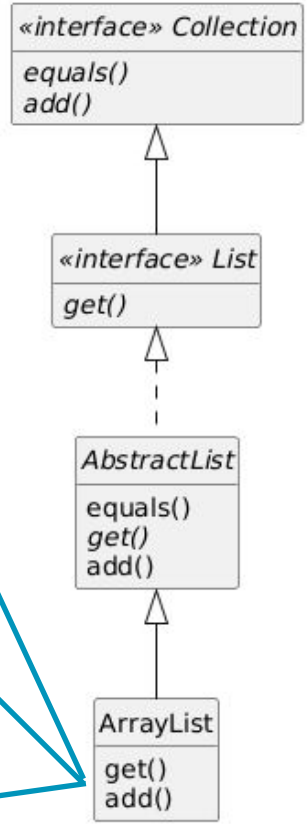
1. A concrete class that implements an interface is required to provide an implementation for all methods of that interface, including the methods that the interface itself inherits from its super interfaces.
2. If a class does not fully meet this requirement, that class must be declared as abstract.
3. A class can implement multiple interfaces (multiple interface inheritance).
4. A class can inherit from at most one other class (single class inheritance).

Interfaces

ArrayList must implement get() because it inherits this method as an abstract method of AbstractList, as shown by the italic style.

Since add() was already implemented by AbstractList, we should consider the presence of add() in ArrayList as overwriting the implementation in AbstractList with our own implementation of ArrayList

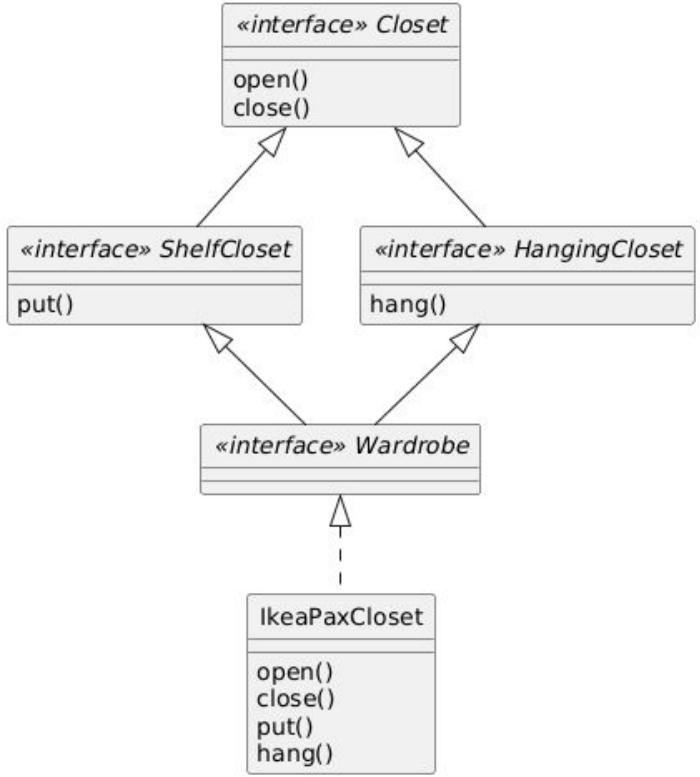
The lack of equals() in ArrayList indicates that ArrayList inherits AbstractList's implementation of equals.



Interfaces

- An interface can be used as a type for a variable, but instances cannot be created from it
  - the variable will contain an object of a concrete subclass
- Which method(s) should the Wardrobe class implement?
- For each of the following statements, please indicate whether it is correct.

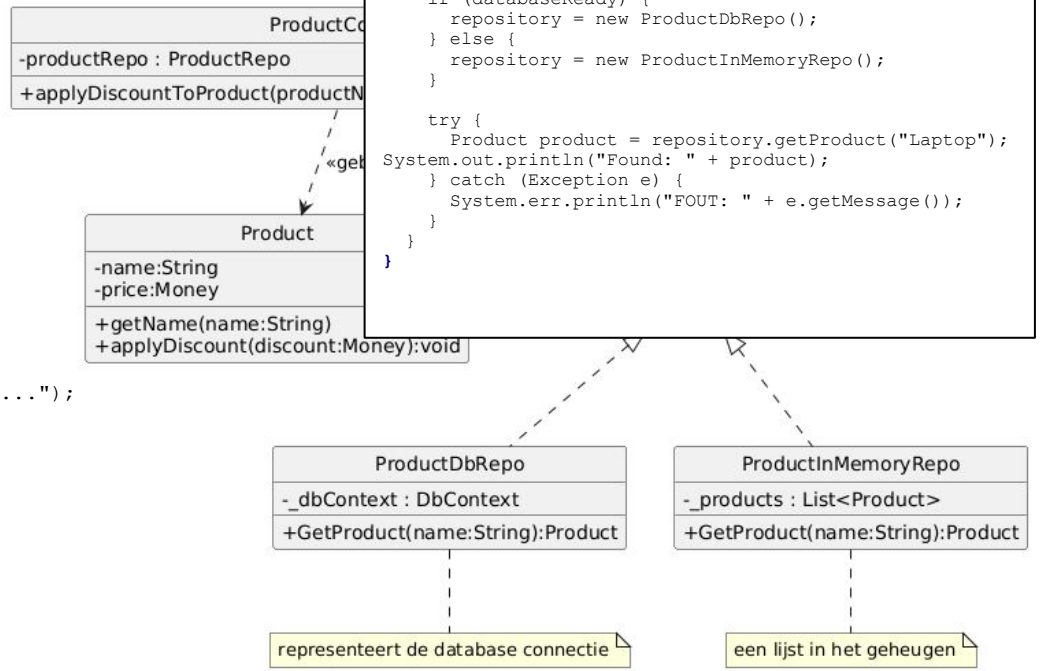
```
Wardrobe myWardrobe = new Wardrobe();
ShelfCloset hisWardrobe = new ShelfCloset();
HangingCloset hairCupboard = new
HangingCloset();
Wardrobe yourCloset = new IkeaPaxCloset();
ShelfCloset urCloset = new IkeaPaxCloset();
HangingCloset theirCloset = new
IkeaPaxCloset();
```



Interfaces and Repo's…

```
interface ProductRepo {
  Product getProduct(String name);
}

class ProductInMemoryRepo implements ProductRepo {
  private final List<Product> products;
  public ProductInMemoryRepo() {
    products = new ArrayList<>();
    products.add(new Product("Laptop", 1200.00));
    products.add(new Product("Muis", 45.50));
  }
  @Override
  public Product getProduct(String name) {
System.out.println("... Searching the list (using a for loop) ...");
    for (Product product : products) {
      if (product.name.equalsIgnoreCase(name)) {
        return product;
      }
    }
    return null;
  }
}

class ProductDbRepo implements ProductRepo {
  @Override
  public Product getProduct(String name) {
System.out.println("... Trying to connect to the database ...");
throw new UnsupportedOperationException("Database is not ready yet!");
  }
}
```

```
public class Demo {
  public static void main(String[] args) {
    boolean databaseReady = false;
    ProductRepo repository;

    if (databaseReady) {
      repository = new ProductDbRepo();
    } else {
      repository = new ProductInMemoryRepo();
    }

    try {
      Product product = repository.getProduct("Laptop");
System.out.println("Found: " + product);
    } catch (Exception e) {
      System.err.println("FOUT: " + e.getMessage());
    }
  }
}
```

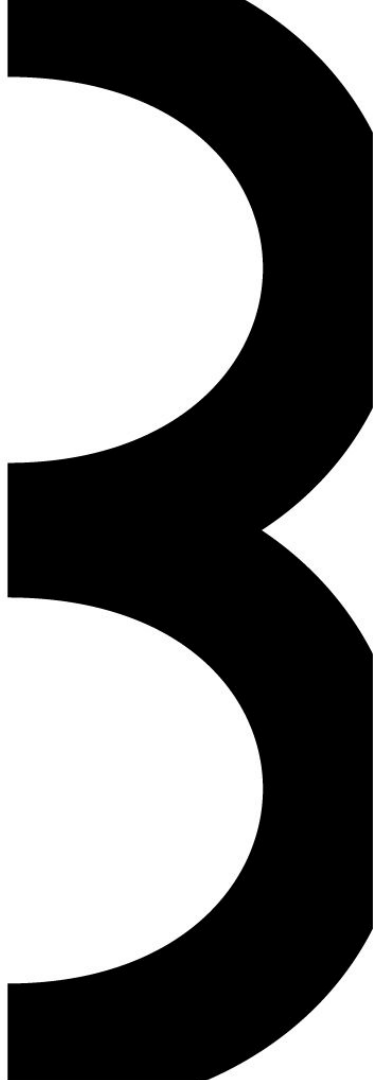


interface = contract

- controller does not need to know anything about the database, only about the contract
- logic testing without a database

cf. socket…

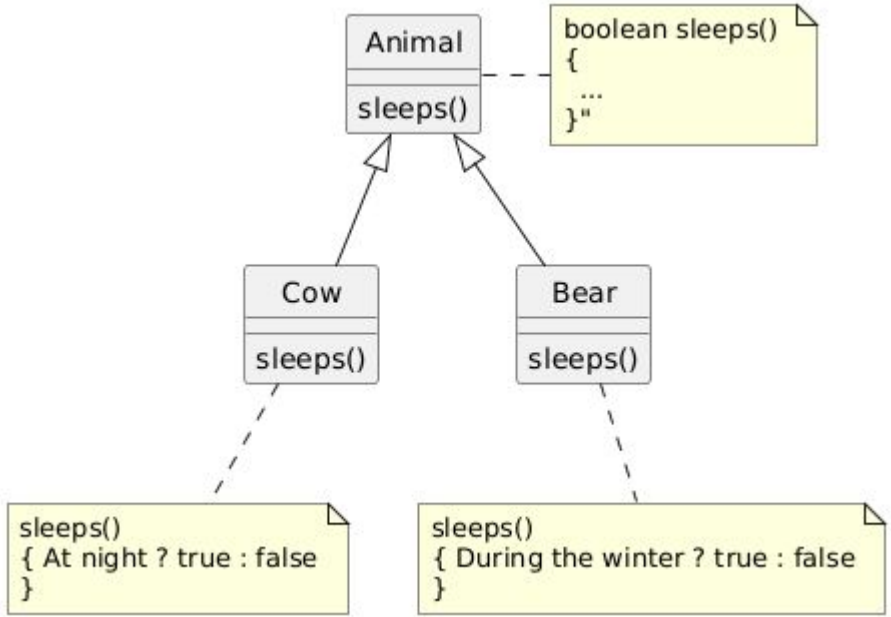
Other additions DCD Polymorphism & inheritance



Polymorphism

```
public class Animal {
 String species;

 public boolean sleeps() {
   ZooTime date=ZooTime.now();
   switch (species) {
     case "cow":
       return date.isNight();
     case "bear":
       return date.isWinter();
     default:
       return false;
   }
 }
}
```

Problem
Information Expert
Low Coupling
High Cohesion



Solution

Polymorphism

```
public class Animal {
 String species;

 public boolean sleeps() {
   ZooTime date=ZooTime.now();
   switch (species) {
     case "cow":
       return date.isNight();
     case "bear":
       return date.isWinter();
     default:
       return false;
   }
 }
}
```

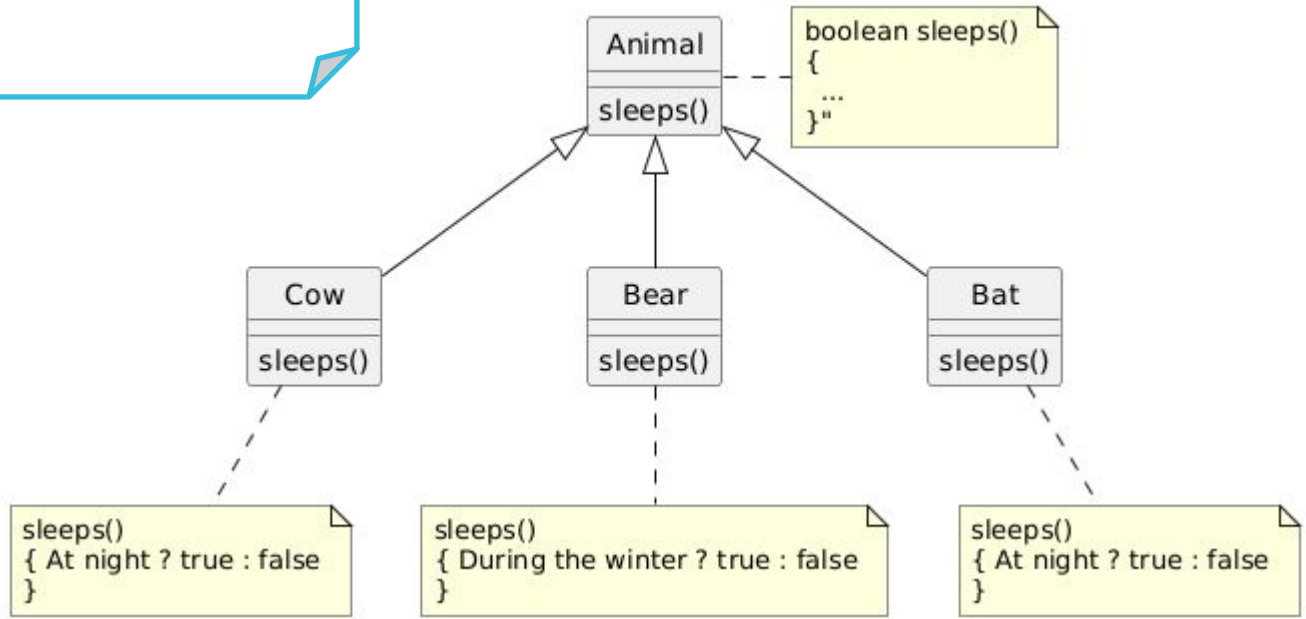
Problem
Information Expert
Low Coupling
High Cohesion

How do we adjust this code when we add a new animal?

Polymorphism

Open Closed principe:
Software must
open to expansions, but closed to changes

Solution + optimization with nocturnal animals?



Inheritance - Liskov Substitution Principle (LSP)

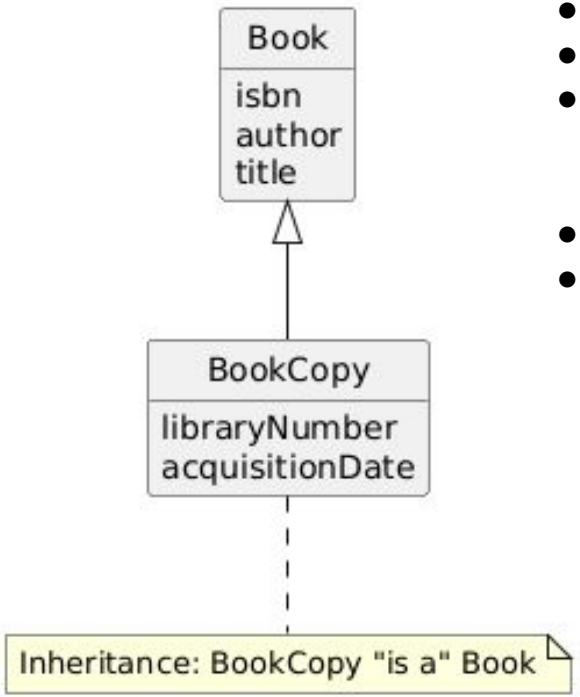
“An object must always be replaceable by an object of a subclass”

- Formulated by Barbara Liskov
- a subclass must support 100% of the attributes and methods of the superclass
- “is a kind of” relationship
- Combination of:
  - 100% rule
  - "is a" rule (see period 1)

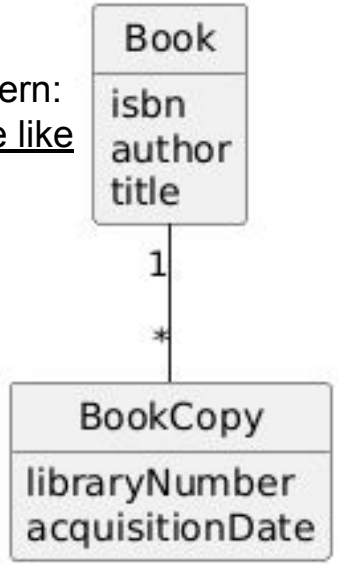
X = subclass
Y = superclass
the question is not "Is an X a kind of Y?",
but: "Does an X always behave like a Y?"

A square is a kind of rectangle, but a square doesn't always behave like a rectangle...

Inheritance - Liskov Substitution Principle (LSP)



- *A book can win a literary prize*
- *A copy may be damaged*
- Book Copy is not 100% in accordance with the superclass associations
  - wrongly modeled generalization
- This is an example of the copy/description pattern:
- CONCLUSION: The superclass cannot behave like its subclass. LSP violated!



Inheritance - Liskov Substitution Principle (LSP)
Design by Contract (Bertrand Meyer)

The requirements (preconditions) must never become stricter. The promises (postconditions) must never become weaker.

A subclass may have more responsibilities, never less. In particular, the subclass may

- not have more presuppositions (not more preconditions)
- not offer fewer guarantees (not fewer postconditions)
- Example: A subclass method in Java may never throw more than one (checked) exception. This changes the method's contract, provides fewer guarantees, and forces the caller to handle more errors.

Inheritance - Liskov Substitution Principle (LSP)
Design by Contract (Bertrand Meyer)

The requirements (preconditions) must never become stricter. The promises (postconditions) must never become weaker.

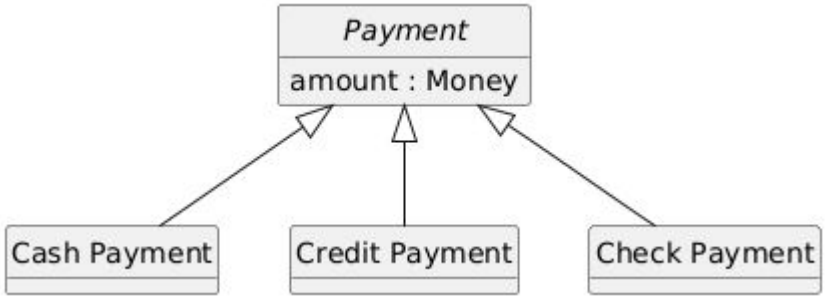
Bertrand Meyer's idea is literally a contract between the caller (the client) and the method (the provider).

- **Preconditions: The client's obligations. What the client must guarantee before calling the method. (e.g., "Give me a number greater than 0")**
- **Postconditions: The provider's obligations. What the method promises to deliver if the client meets the preconditions. (e.g., "I promise to return the square root of that number")**

Rule 1: Preconditions may NOT be made stricter (they can only be relaxed). The subclass may not make more demands on the client.
e.g. Error: Suddenly requiring the number to be an integer (this is a stricter requirement).

Rule 2: Postconditions may NOT be weakened (they may only be strengthened). The subclass must promise at least as much as the superclass, and may promise more.
e.g. Error: Rounding the square root (this is a weaker promise, because it is less precise).

Inheritance

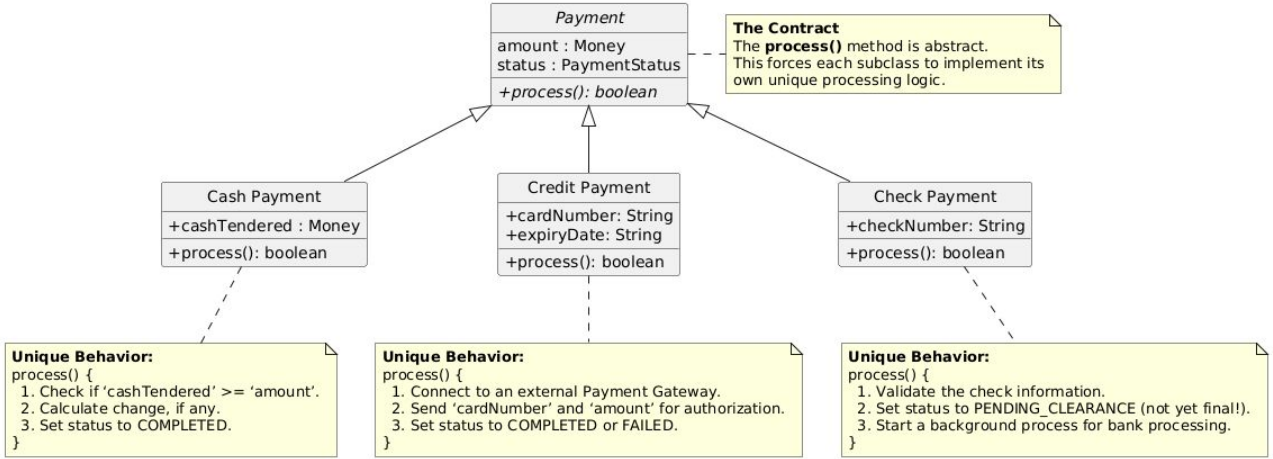


What is the difference with the Animal class (slide 19)?

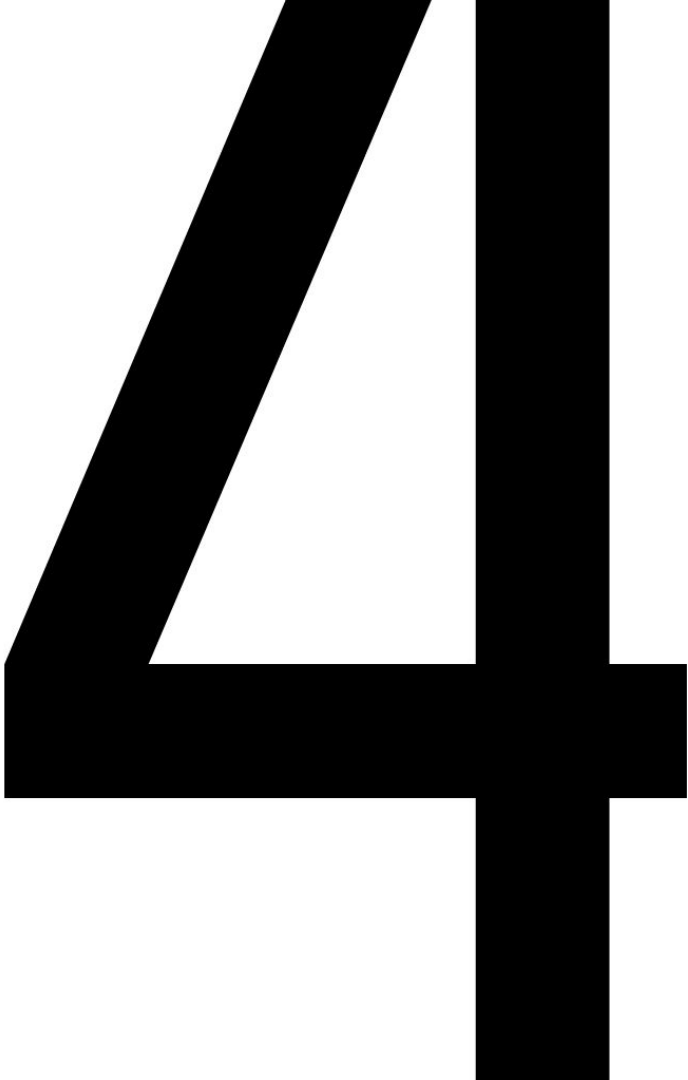
- Rather overkill
- Does the subclass inherit enough to justify inheritance?
- If not, an Enumeration might be simpler

The key is the behavior…

- no overkill
- inheritance justified

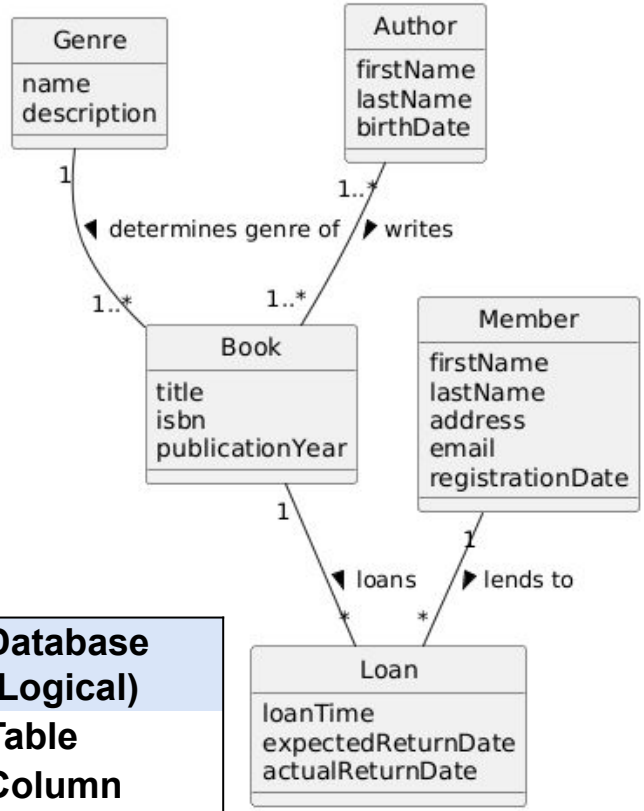


PREVIEW INTEGRATION 1



Checklist: CCD (Conceptual Class Diagram) to ERD

1. **Laying foundations (conventions)**
   - Determining naming conventions for tables and columns
   - Choosing a PK strategy (surrogate keys)
   - Legend for composing keys: PK, FK
2. **Creating tables and columns (basic structure)**
   - Each Class → becomes a Table
   - Each Attribute → becomes a Column
3. **Building structure and relationships (connections)**
   - Add Primary Keys (PK) to each table
   - **1-to-many relationship → add FK to the "many" side**
   - **Many-to-many relationship → create intermediate table with PK,FK's**
4. **Completing the diagram (finalization)**
   - Drawing relationship lines
   - Adding descriptive labels to relationships
   - Final check and consistency check



Domain Model (Conceptual)	→	Database (Logical)
Class	→	Table
Attribute	→	Column
Attribute value	→	Field
Association	→	Relation

Checklist: CCD (Conceptual Class Diagram) to ERD

1. Laying foundations (conventions)

- **Determine naming convention:**
  Tables: used style = UPPERCASE_SNAKE_CASE and in plural
  e.g. class 'Member' becomes table 'MEMBERS'
  Columns: used style = lowercase_snake_case
  e.g. attribute 'dateReservation' becomes column 'date_reservation'
- **Add Primary Keys (PK):**
  Add **a surrogate key** (e.g., member_id, book_id) to each table. Mark it as a PK. This means that each table gets its own artificial identifier column that has nothing to do with the real world.
- **Visual convention for capturing constraints:**
  This is the legend we'll use:
  PK: primary key
  FK: foreign key

Checklist: CCD (Conceptual Class Diagram) to ERD

2. Creating tables and columns (basic structure)
Translate the basic building blocks of your domain model.

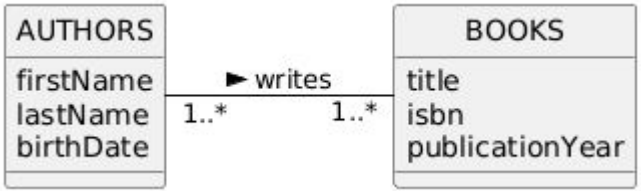
- **Class → Table:**
  Create a table for each class in the domain model using the naming convention
- **Attribute → Column:**
  Copy each attribute of the class as a column in the corresponding table, also according to the naming convention

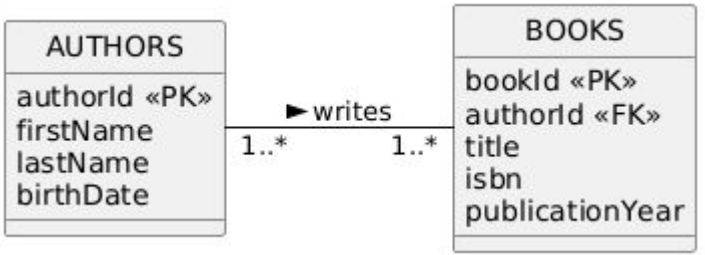
Checklist: CCD (Conceptual Class Diagram) to ERD

3. Building structure and relationships (connections)

Build the backbone of the database: the identity of each table and the connections between them.

- **One-to-many relationships (1..*) translate:**
  Take the HP from the table on the "one" side.
  Add this as a new column in the table on the "many" side and mark as FK.



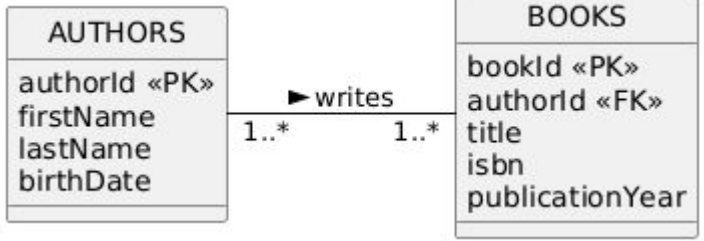


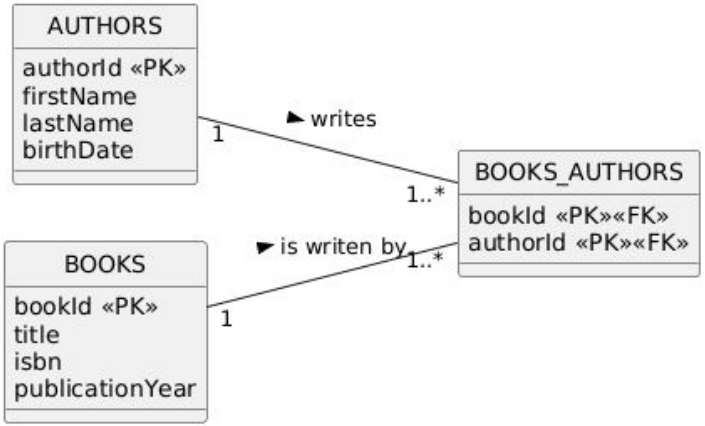
Checklist: CCD (Conceptual Class Diagram) to ERD

3. Building structure and relationships (connections)

Build the backbone of the database: the identity of each table and the connections between them.

- **Many-to-many Relationships (..) Translate:**
  Create a new junction table.
  Take the HPs from both original tables and add them as columns to this new table.
  Mark both columns as both PK and FK.



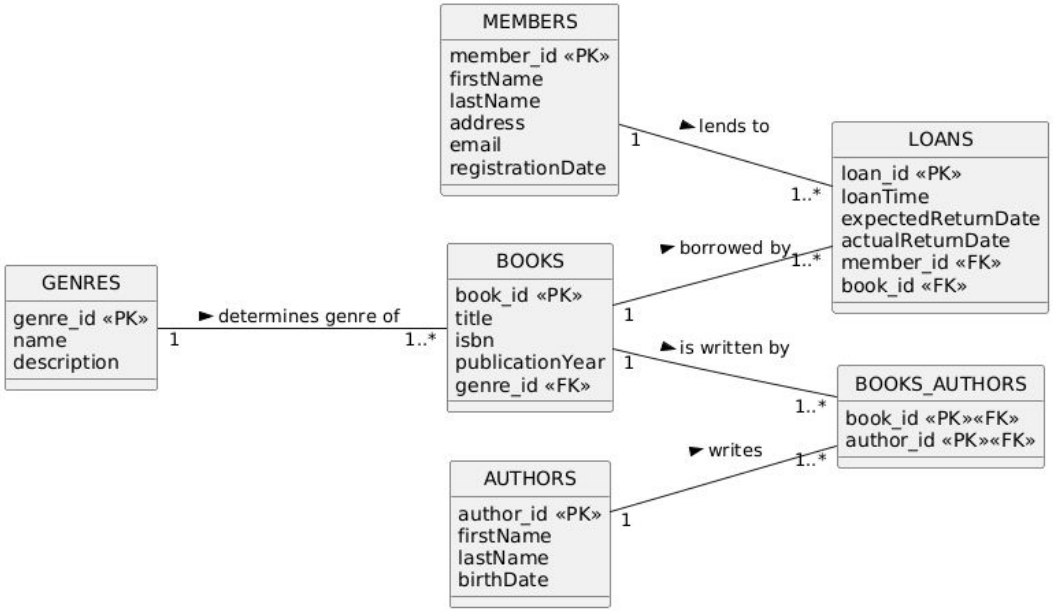


Checklist: CCD (Conceptual Class Diagram) to ERD

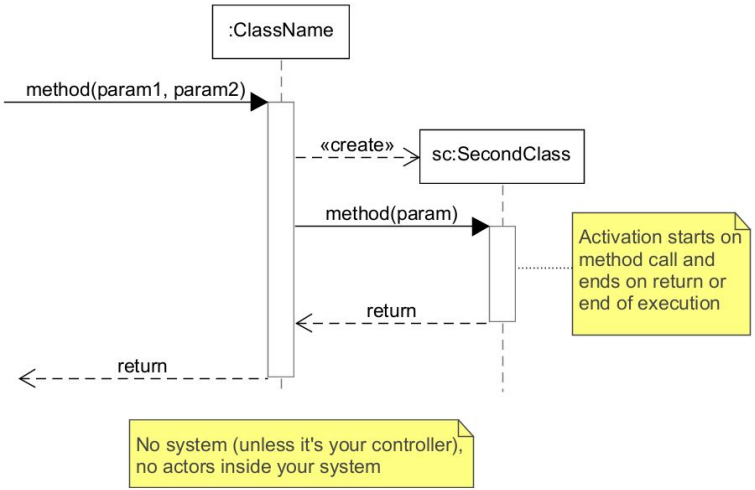
4. Completing the diagram (finalization)
Make the ERD complete and readable.

- **Review and revise: Review the entire diagram. Is the naming consistent? Does each FK refer to an existing PK? Is the legend clear? Is the diagram clear and correct according to the established rules?**

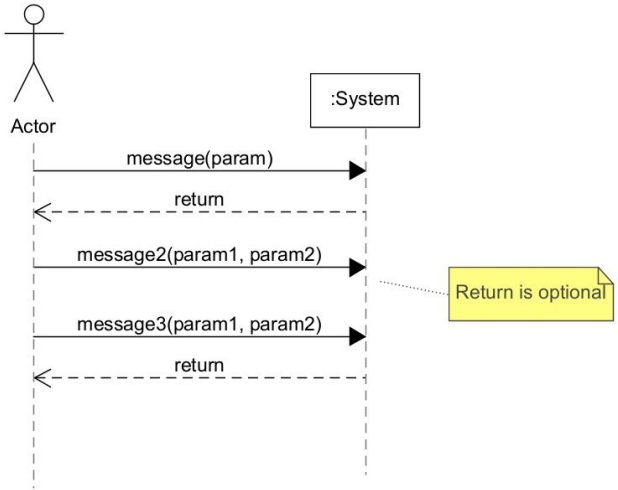
PK: primary key
FK: foreign key



Sequence Diagram

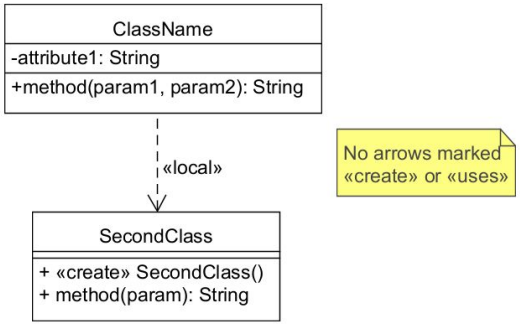


System Sequence Diagram



Term 2 delivery expectations (minimum):
Everything from P1 +
- 2 System Sequence Diagrams
- 4 Operational Contracts (2 per SSD)
- 4 Sequence Diagrams (1 per OC)
- 1 Design Class Diagram

Design Class Diagram



Note: You must submit your portfolio to the week 11 assignment **individually** on time to participate in the exam!

KdG Karel de Grote Hogeschool